



دانشکده مهندسی کامپیوتر

دانشگاه صنعتی شریف

آزمون پذیری

فاز اول پروژه

نام و نام خانوادگی:

مهران تقوی

استاد :

دکتر حسابی

پاییز ۱۴۰۳

گام اول پروژه:

صورت مسئله:

استفاده از زبان‌های برنامه نویسی یک شبیه‌ساز طراحی و پیاده‌سازی نمایید که فایل‌های استاندارد ISCAS را بخواند و این شبیه‌ساز بایستی قادر باشد که با مقادیر ورودی که به مدار اعمال می‌کند، مقادیر تمام نت‌ها و خروجی‌ها را تحویل دهد. برای ایجاد شبیه‌ساز بایستی فایل isc را خوانده و یکبار بدون تاخیر و یکبار هم با داشتن تاخیر خروجی را تولید کند.

پیاده‌سازی:

برای پیاده‌سازی این بخش باید پکیج‌های مورد نیاز و ضروری را اضافه کنیم.

```
from enum import Enum
```

مقادیر پذیرفته شده را برای سیگنال‌هایمان تعریف می‌کنیم:

```
class ValueType(Enum):  
    One = "1"  
    Zero = "0"  
    Unknown = "U"  
    HighZ = "Z"
```

حال یک کلاس به نام Testability استفاده کرده‌ایم.

```
class Testability:  
    operations = {}  
  
    def addOperation(self, name : str, operation):  
        self.name = name  
        self.operations[name] = operation  
  
    def getValue(self, operation_name : str, inputs : list[ValueType]):  
        operation = self.operations.get(operation_name)  
        return operation(inputs)
```

حال یک تابع به نام readISC استفاده کرده ایم:

تابع readISC که کد آن در زیر نیز آورده شده است، فایل isc را بصورت خط به خط خوانده و در یک لیست ذخیره می کند و همچنین خطوطی را که با * شروع شده اند (خطوط comment) را اصلا نمی خواند. به عبارتی ما یک لیست به نام lines داریم که هر عنصر این لیست، یک لیستی حاوی یک خط از فایل isc است. همچنین مقادیر تمام خطوط مدار را بصورت پیش فرض برابر ۱ قرار داده ایم.

```
def readISC(self, file) -> list:
    netlist = {}
    isInputsLine = False
    lastNet = None
    for line in file.readlines():
        if line.startswith('*'):
            continue
        tmp = line.strip().split()

        if isInputsLine:
            tmp = netlist[lastNet] + tmp
            tmp.append(ValueType.Unknown)
            isInputsLine = False

        elif tmp[2] == "inpt" or tmp[2] == "from":
            tmp.append(ValueType.Unknown)

        else:
            isInputsLine = True
            lastNet = tmp[0]

    netlist[tmp[0]] = tmp
    return netlist
```

توابع andGate، orGate، nandGate، norGate، xorGate و xnorGate به ترتیب منطق گیت های and، or، nand، nor، xor و xnor را پیاده سازی می کند. به عنوان مثال در کد زیر که تابع andGate را نشان می دهد.

```
def andGate(inputs : list[ValueType]):
    if all(input == ValueType.One for input in inputs):
        return ValueType.One
    elif any(input == ValueType.Zero for input in inputs):
        return ValueType.Zero
    elif ValueType.Unknown in inputs:
        return ValueType.Unknown
    else:
        return ValueType.HighZ

obj.addOperation("and", andGate);
```

همان طور که در کد بالا قابل مشاهده است می توان برای دو ورودی تست کرد:

- در صورتی که هر دو ورودی گیت and مقدار 1 داشته باشند، خروجی نیز 1 می‌شود.
 - در صورتی که حداقل یکی از ورودیهای گیت and صفر باشد خروجی صفر خواهد شد.
 - در صورتی که حداقل یکی از ورودیهای گیت and مقدار 1 داشته باشند و دیگری مقدار 1 یا 0 داشته باشد، خروجی نیز 1 می‌شود.
 - در صورتی که موارد بالا رخ ندهند، خروجی برابر HighZ می‌شود.
- همچنین برای گیت‌های دیگر هم کدهای مورد نظر را در شکل‌های زیر مشاهده می‌کنید:

:OrGate

```
def orGate(inputs : list[ValueType]):
    if any(input == ValueType.One for input in inputs):
        return ValueType.One
    elif all(input == ValueType.Zero for input in inputs):
        return ValueType.Zero
    elif ValueType.Unknown in inputs:
        return ValueType.Unknown
    else:
        return ValueType.HighZ

obj.addOperation("or", orGate);
```

:NandGate

```
def nandGate(inputs : list[ValueType]):
    if all(input == ValueType.One for input in inputs):
        return ValueType.Zero
    elif any(input == ValueType.Zero for input in inputs):
        return ValueType.One
    elif ValueType.Unknown in inputs:
        return ValueType.Unknown
    else:
        return ValueType.HighZ

obj.addOperation("nand", nandGate);
```

:NorGate

```
def norGate(inputs : list[ValueType]):
    if any(input == ValueType.One for input in inputs):
        return ValueType.Zero
    elif all(input == ValueType.Zero for input in inputs):
        return ValueType.One
    elif ValueType.Unknown in inputs:
        return ValueType.Unknown
    else:
        return ValueType.HighZ

obj.addOperation("nor", norGate);
```

:XorGate

```
def xorGate(inputs : list[ValueType]):
    if any(input == ValueType.HighZ for input in inputs):
        return ValueType.HighZ
    if any(input == ValueType.Unknown for input in inputs):
        return ValueType.Unknown
    else:
        if inputs.count(ValueType.One) % 2 == 1:
            return ValueType.One
        else:
            return ValueType.Zero

obj.addOperation("xor", xorGate);
```

:XnorGate

```
def xnorGate(inputs : list[ValueType]):
    if any(input == ValueType.HighZ for input in inputs):
        return ValueType.HighZ
    if any(input == ValueType.Unknown for input in inputs):
        return ValueType.Unknown
    else:
        if inputs.count(ValueType.One) % 2 == 1:
            return ValueType.Zero
        else:
            return ValueType.One

obj.addOperation("xnor", xnorGate);
```

:NotGate

```
def notGate(inputs : list[ValueType]):
    if inputs[0] == ValueType.HighZ:
        return ValueType.HighZ
    if inputs[0] == ValueType.One:
        return ValueType.Zero
    elif inputs[0] == ValueType.Zero:
        return ValueType.One
    else:
        return ValueType.Unknown

obj.addOperation("not", notGate);
```

:BufferGate

```
def buffGate(inputs : list[ValueType]):
    return inputs[0]

obj.addOperation("buff", buffGate);
obj.addOperation("buf", buffGate);
```

با استفاده از تابع `initialValue` که کد آن در زیر نیز قابل مشاهده است، مقداردهی اولیه را انجام می‌دهیم. در واقع در این تابع خطوطی که نقش ورودی مدار را دارند، مقداردهی می‌شوند.

```
def initialValue(self, netList, inputValue):
    for i in inputValue.keys():
        net = netList[i]
        if net[2] == 'inpt':
            net[len(net) - 1] = inputValue[i]
```

تابع `readInputFile` که در زیر نیز آورده شده است، فایل ورودی را خط به خط خوانده و مقادیری که به عنوان ورودی در آن فایل قرار داشتند را در دیکشنری به نام `wire_values` می‌ریزد. و مقادیر زمان را هم در لیستی به نام `time_values` قرار می‌دهد

```
def readInputFile(self, input):
    header = input.readline().strip().split()
    wire_names = header[0].replace("Input:", "")
    wire_names = [wire_names] + header[1:-1]
    wire_values = {wire: [] for wire in wire_names}
    time_values = []

    for line in input.readlines():
        values = line.strip().split()
        time = int(values[-1])
        time_values.append(time)

        for wire_name, value in zip(wire_names, values[:-1]):
            if value == "1":
                wire_values[wire_name].append(ValueType.One)
            elif value == "0":
                wire_values[wire_name].append(ValueType.Zero)
            elif value == "U":
                wire_values[wire_name].append(ValueType.Unknown)
            elif value == "Z":
                wire_values[wire_name].append(ValueType.HighZ)
    return wire_values, time_values
```

تابع `getValue` که در تصویر زیر قابل مشاهده است

```
def getValue(self, operation_name : str, inputs : list[ValueType]):
    operation = self.operations.get(operation_name)
    return operation(inputs)
```

تابع `exportValues` به صورت زیر است که فایل ورودی و فایل `isc` را دریافت می‌کند و مقدار همه سیگنال‌ها را محاسبه کرده و مقادیر خروجی را برای زمانهای مختلف با ورودی‌های متفاوت در حالت بدون تاخیر در فایل خروجی ذخیره می‌کند.

```
def exportValues(inputFilePath: str, iscFilePath: str, outputFilePath: str):
    with open(inputFilePath, 'r') as inputFile:
        with open(iscFilePath, 'r') as iscFile:
            netList = obj.readISC(iscFile)
            wire_values, time_values = obj.readInputFile(inputFile)
            with open(outputFilePath, "w") as outF:
                for t, time in enumerate(time_values):
                    inputValue = {wire_name: wire_values[wire_name][t] for wire_name in wire_values}
                    obj.initialValue(netList, inputValue)
                    obj.assignAllValues(netList)
                    outF.write(f"Time: {time}\n" + "*" * 50 + "\n")
                    for i in netList.keys():
                        net = netList[i]
                        outF.write(f"{net[0]}\t\t{net[-1].value}\n")
                    outF.write("\n")
```

طبق کد، فایلی که مقادیر ورودی در آن قرار دارند را خوانده و همچنین فایل isc مورد نظر را نیز خوانده سپس براساس فایل isc خوانده شده netlist را ایجاد کرده و همچنین براساس فایل حاوی مقادیر ورودی، inputValue را ایجاد کرده و در آخر براساس netlist و inputValue با استفاده از تابع initialValue، مدار را مقداردهی اولیه می کنیم.

تابع assignValue که در تصویر زیر قابل مشاهده است، که مقدار سیگنال ها را در netlist تعیین می کند.

سپس طبق کد زیر، براساس netlist ایجاد شده و مقداردهی اولیه انجام شده، بقیه ی خطوط مدار با استفاده از تابع getValue مقداردهی می شوند و پس از آن در فایل خروجی نوشته میشوند.

```
def assignAllValues(self, netList):
    for i in netList.keys():
        net = netList[i]
        if net[2] == 'inpt':
            continue
        elif net[2] == 'from':
            for j in netList.keys():
                if netList[j][1] == net[3]:
                    inputNet = netList[j]
                    net[len(net) - 1] = inputNet[len(inputNet) - 1]
                    break
            else:
                startInputsIndex = len(net) - 1 - int(net[4])
                endInputsIndex = len(net) - 1
                inputNets = [netList[j] for j in net[startInputsIndex: endInputsIndex]]
                inputValues = [inputNet[len(inputNet) - 1] for inputNet in inputNets]
                net[len(net) - 1] = self.getValue(net[2], inputValues)
```

نتایج:

اجرای کد با c1.isc:

```
*c17 iscas example (to test conversion program only)
*-----
*
*
* total number of lines in the netlist .....      17
* simplistically reduced equivalent fault set size =    22
*   lines from primary input gates .....          5
*   lines from primary output gates .....         2
*   lines from interior gate outputs .....        4
*   lines from **      3 ** fanout stems ...       6
*
* avg_fanin  = 2.00,     max_fanin  = 2
* avg_fanout = 2.00,     max_fanout = 2
*
*
*
*
*
*
*
1      1gat inpt      1    0      >sa1
2      2gat inpt      1    0      >sa1
3      3gat nand      0    2      >sa1
|      |
1      2
```

حال در فایل c1_inputs_values.txt مقادیر ورودی در زمان های مختلف را بصورت زیر قرار می دهیم:

Input	1	2	Time
	1	0	0
	1	1	1
	1	0	2
	1	1	3

پس از اجرای کد خروجی در فایل `cl_net_values.txt` بصورت زیر ذخیره می‌شود:

Time: 1	Time: 2	Time: 3	Time: 0
*****	*****	*****	*****
1 1	1 1	1 1	1 1
2 1	2 U	2 Z	2 0
3 0	3 U	3 Z	3 1

```
*c17 iscas example (to test conversion program only)
*-----
*
*
* total number of lines in the netlist ..... 17
* simplistically reduced equivalent fault set size = 22
*   lines from primary input gates ..... 5
*   lines from primary output gates ..... 2
*   lines from interior gate outputs ..... 4
*   lines from **      3      ** fanout stems ... 6
*
*   avg_fanin  = 2.00,    max_fanin  = 2
*   avg_fanout = 2.00,    max_fanout = 2
*
*
*
*
*
*
1       1gat inpt     1   0       >sa1
2       2gat inpt     1   0       >sa1
3       3gat nand      1   2       >sa1
| 1         2
4       4gat inpt     1   0       >sa1
5       5gat and       0   2       >sa1
| 3         4
```

حال در فایل c5_inputs_values.txt مقادیر ورودی در زمان های مختلف را بصورت زیر قرار می دهیم:

Input:	1	2	4	Time
	1	0	1	0
	1	1	0	1
	1	0	0	2
	1	1	1	3

پس از اجرای کد خروجی در فایل c5_net_values.txt بصورت زیر ذخیره می‌شود:

Time: 1		Time: 2		Time: 3		Time: 0	
*****		*****		*****		*****	
1	1	1	1	1	1	1	1
2	1	2	0	2	Z	2	0
3	0	3	1	3	Z	3	1
4	0	4	U	4	1	4	1
5	0	5	U	5	Z	5	1

اجرای کد با c17.isc:

```

1 1gat inpt 1 0 >sa1
2 2gat inpt 1 0 >sa1
3 3gat inpt 2 0 >sa0 >sa1
8 8fan from 3gat >sa1
9 9fan from 3gat >sa1
6 6gat inpt 1 0 >sa1
7 7gat inpt 1 0 >sa1
10 10gat nand 1 2 >sa1
1 8
11 11gat nand 2 2 >sa0 >sa1
9 6
14 14fan from 11gat >sa1
15 15fan from 11gat >sa1
16 16gat nand 2 2 >sa0 >sa1
2 14
20 20fan from 16gat >sa1
21 21fan from 16gat >sa1
19 19gat nand 1 2 >sa1
15 7
22 22gat nand 0 2 >sa0 >sa1
10 20
23 23gat nand 0 2 >sa0 >sa1
21 19

```

حال در فایل c17_inputs_values.txt مقادیر ورودی در زمان های مختلف را بصورت زیر قرار می دهیم:

```

Input:1 3 6 2 7 Time
      1 0 1 0 0 0
      1 1 1 0 0 1

```

پس از اجرای کد خروجی در فایل c17_net_values.txt بصورت زیر ذخیره می شود:

```

Time: 1          Time: 0
*****          *****
1          1      1          1
2          0      2          0
3          1      3          0
8          1      8          0
9          1      9          0
6          1      6          1
7          0      7          0
10         0      10         1
11         0      11         1
14         0      14         1
15         0      15         1
16         1      16         1
20         1      20         1
21         1      21         1
19         1      19         1
22         1      22         0
23         0      23         0

```

برای فایل c432.isc داریم که قسمتی از این فایل در عکس زیر آمده است:

```

222 104
260 260gat nand 2 2 >sa0 >sa1
225 157
261 260f01 from 260gat >sa1
262 260f02 from 260gat >sa0 >sa1
263 263gat nand 1 2
226 158
264 264gat nand 2 2 >sa0 >sa1
228 183
265 264f01 from 264gat >sa1
266 264f02 from 264gat >sa0 >sa1
267 267gat nand 2 2 >sa0 >sa1
231 185
268 267f01 from 267gat >sa1
269 267f02 from 267gat >sa0 >sa1
270 270gat nand 2 2 >sa0 >sa1
234 187
271 270f01 from 270gat >sa1
272 270f02 from 270gat >sa0 >sa1
273 273gat nand 2 2 >sa0 >sa1
237 189
274 273f01 from 273gat >sa1
275 273f02 from 273gat >sa0 >sa1
276 276gat nand 2 2 >sa0 >sa1
240 191
277 276f01 from 276gat >sa1
278 276f02 from 276gat >sa0 >sa1
279 279gat nand 2 2 >sa0 >sa1
244 193
280 279f01 from 279gat >sa1

```

حال در فایل c432_inputs_values.txt مقادیر ورودی در یک زمان را بصورت زیر قرار می‌دهیم:

```

Input:1 4 8 11 14 17 21 24 27 30 34 37 40 43 47 50 53 56 60 63 66 69 73 76 79 82 86 89 92 95 99 102 105 108 112
      1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1 1 0 0 0 0 0 1 0 0 1 0 0 1 0

```

پس از اجرای کد خروجی در فایل c432_net_values.txt بصورت زیر ذخیره می‌شود. که بخشی از آن در زیر آمده است

61	1	30	0	Time: 0	
62	1	31	0	*****	
63	0	32	0	1	1
64	0	33	0	2	1
65	0	34	1	3	1
66	1	35	1	4	0
67	1	36	1	5	0
68	1	37	0	6	0
69	1	38	0	7	0
70	1	39	0	8	1
71	1	40	1	9	1
72	1	41	1	10	1
73	1	42	1	11	0
74	1	43	0	12	0
75	1	44	0	13	0
76	1	45	0	14	1
77	1	46	0	15	1
78	1	47	1	16	1
79	0	48	1	17	0
80	0	49	1	18	0
81	0	50	0	19	0
82	0	51	0	20	0
83	0	52	0	21	1
84	0	53	1	22	1
85	0	54	1	23	1
86	0	55	1	24	0
87	0	56	0	25	0
88	0	57	0	26	0
89	0	58	0	27	1
90	0	59	0	28	1
91	0	60	1	29	1
92	0	61	1		

کد زیر اضافه شده است که توسط آن مقدار حداکثر زمان مورد نیاز برای valid شدن همه ی خروجی ها نیاز است محاسبه میشود که مقدار آن برابر تاخیر مسیر بحرانی بعلاوه آخرین زمان ورودی است تا مدار به حالت پایدار برسد.

```
def calculateMaxTime(self, netList_copy):
    outputNetDelay = []
    for i in netList_copy.keys():
        net = netList_copy[i]

        if net[2] == 'inpt' or net[2] == 'from':
            continue
        else:
            startInputsIndex = len(net) - 2 - int(net[4])
            endInputsIndex = len(net) - 2
            inputNets = [netList_copy[j] for j in net[startInputsIndex: endInputsIndex]]
            inputDelays = []
            for inputNet in inputNets:
                if inputNet[2] == 'inpt':
                    inputDelays.append(0)
                elif inputNet[2] == 'from':
                    x = netList_copy[inputNet[3].replace("gat", "")]
                    if x[2] == 'inpt':
                        inputDelays.append(0)
                    else:
                        inputDelays.append(int(x[-2]))
                else:
                    inputDelays.append(int(inputNet[-2]))
            netDelay = max(inputDelays, default=0)
            net[-2] = int(net[-2]) + netDelay
            if net[3] == '0':
                outputNetDelay.append(int(net[-2]))
            print("Delay:", max(outputNetDelay))
    return max(outputNetDelay)
```

برای محاسبه c17 به صورت دستی انجام دادیم که نتیجه آن در زیر آمده است:

Net time	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵	۱۶	۱۷	۱۸	۱۹	۲۰	۲۱	۲۲	۲۳
۰	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۲	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۳	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۴	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۵	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۶	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۷	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۸	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۹	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۱۰	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۱۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱
۱۲	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱	۱

اجرای کد با c1.isc با اضافه کردن تاخیر:

```
*c17 iscas example (to test conversion program only)
*-----
*
*
* total number of lines in the netlist .....      17
* simplistically reduced equivalent fault set size =    22
*     lines from primary input gates .....          5
*     lines from primary output gates .....           2
*     lines from interior gate outputs .....           4
*     lines from **      3      ** fanout stems ...   6
*
*         avg_fanin = 2.00,       max_fanin = 2
*         avg_fanout = 2.00,      max_fanout = 2
*
*
*
*
*
*
*

```

1	1gat inpt	1	0	>sa1
2	2gat inpt	1	0	>sa1
3	3gat nand	0	2	>sa1
1	2	2		

در فایل c1_inputs_values.txt مقادیر ورودی در زمان های مختلف را بصورت زیر قرار می دهیم:

Input	1	2	Time
	1	0	0
	1	1	1
	1	U	2
	1	Z	3

پس از اجرای کد خروجی در فایل `cl_net_values.txt` بصورت زیر ذخیره می‌شود:

Time: 1		Time: 2		Time: 3		Time: 4		Time: 5		Time: 6	
*****		*****		*****		*****		*****		*****	
1	1	1	1	1	1	1	1	1	1	1	1
2	1	2	U	2	Z	2	Z	2	Z	2	0
3	U	3	1	3	0	3	U	3	Z	3	U

اجرای کد با c17.isc با اضافه کردن تاخیر:

```
*
1 1gat inpt 1 0 >sa1
2 2gat inpt 1 0 >sa1
3 3gat inpt 2 0 >sa0 >sa1
8 8fan from 3gat >sa1
9 9fan from 3gat >sa1
6 6gat inpt 1 0 >sa1
7 7gat inpt 1 0 >sa1
10 10gat nand 1 2 >sa1
1 8 2
11 11gat nand 2 2 >sa0 >sa1
9 6 3
14 14fan from 11gat >sa1
15 15fan from 11gat >sa1
16 16gat nand 2 2 >sa0 >sa1
2 14 3
20 20fan from 16gat >sa1
21 21fan from 16gat >sa1
19 19gat nand 1 2 >sa1
15 7 3
22 22gat nand 0 2 >sa0 >sa1
10 20 5
23 23gat nand 0 2 >sa0 >sa1
21 19 3
```

حال در فایل c17_inputs_values.txt مقادیر ورودی در زمان های مختلف را بصورت زیر قرار می دهیم:

```
Input:1 3 6 2 7 Time
1 1 0 0 0 0
0 0 1 0 1 1
```

پس از اجرای کد خروجی در فایل c17_net_values.txt بصورت زیر ذخیره می شود، که بخشی از آن را در پایین مشاهده میکنید.

Time: 1	Time: 2	Time: 3	Time: 4	Time: 5	Time: 0
*****	*****	*****	*****	*****	*****
1 0	1 0	1 0	1 0	1 0	1 1
2 0	2 0	2 0	2 0	2 0	2 0
3 0	3 0	3 0	3 0	3 0	3 1
8 0	8 0	8 0	8 0	8 0	8 1
9 0	9 0	9 0	9 0	9 0	9 1
6 1	6 1	6 1	6 1	6 1	6 0
7 1	7 1	7 1	7 1	7 1	7 0
10 U	10 0	10 1	10 1	10 1	10 U
11 U	11 U	11 1	11 1	11 1	11 U
14 U	14 U	14 1	14 1	14 1	14 U
15 U	15 U	15 1	15 1	15 1	15 U
16 U	16 U	16 1	16 1	16 1	16 U
20 U	20 U	20 1	20 1	20 1	20 U
21 U	21 U	21 1	21 1	21 1	21 U
19 U	19 U	19 1	19 U	19 U	19 U
22 U	22 U	22 U	22 U	22 U	22 U
23 U	23 U	23 U	23 U	23 U	23 U

برای فایل c432.isc داریم که قسمتی از این فایل در عکس زیر آمده است:

268	267f01	from	267gat	>sa1	225	224f01	from	224gat	>sa1
269	267f02	from	267gat	>sa0 >sa1	226	224f02	from	224gat	>sa1
270	270gat	nand	2 2	>sa0 >sa1	227	227gat	xor	2 2	>sa0 >sa1
234	187	2			205	161	2		
271	270f01	from	270gat	>sa1	228	227f01	from	227gat	>sa1
272	270f02	from	270gat	>sa0 >sa1	229	227f02	from	227gat	>sa1
273	273gat	nand	2 2	>sa0 >sa1	230	230gat	xor	2 2	>sa0 >sa1
237	189	2			206	164	2		
274	273f01	from	273gat	>sa1	231	230f01	from	230gat	>sa1
275	273f02	from	273gat	>sa0 >sa1	232	230f02	from	230gat	>sa1
276	276gat	nand	2 2	>sa0 >sa1	233	233gat	xor	2 2	>sa0 >sa1
240	191	2			207	167	5		
277	276f01	from	276gat	>sa1	234	233f01	from	233gat	>sa1
278	276f02	from	276gat	>sa0 >sa1	235	233f02	from	233gat	>sa1
279	279gat	nand	2 2	>sa0 >sa1	236	236gat	xor	2 2	>sa0 >sa1
244	193	2			208	170	5		
280	279f01	from	279gat	>sa1	237	236f01	from	236gat	>sa1
281	279f02	from	279gat	>sa0 >sa1	238	236f02	from	236gat	>sa1
282	282gat	nand	2 2	>sa0 >sa1	239	239gat	xor	2 2	>sa0 >sa1
248	195	2			209	173	5		
283	282f01	from	282gat	>sa1	240	239f01	from	239gat	>sa1
284	282f02	from	282gat	>sa0 >sa1	241	239f02	from	239gat	>sa1
285	285gat	nand	2 2	>sa0 >sa1	242	242gat	nand	1 2	>sa1
252	197	2			3	214	5		
286	285f01	from	285gat	>sa1	243	243gat	xor	2 2	>sa0 >sa1
287	285f02	from	285gat	>sa0 >sa1	210	176	5		
288	288gat	nand	1 2		244	243f01	from	243gat	>sa1
229	184	2			245	243f02	from	243gat	>sa1
289	289gat	nand	1 2		246	246gat	nand	1 2	>sa1
232	186	2			215	13	5		

حال در فایل c432_inputs_values.txt مقادیر ورودی در یک زمان را بصورت زیر قرار می‌دهیم:

```
Input:1 4 8 11 14 17 21 24 27 30 34 37 40 43 47 50 53 56 60 63 66 69 73 76 79 82 86 89 92 95 99 102 105 108 112
      1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1 1 0 0 0 0 0 1 0 0 1 0 0 1 0
```

پس از اجرای کد خروجی در فایل c432_net_values.txt بصورت زیر ذخیره می‌شود. که بخشی از آن در زیر آمده است:

50	48	1	1741	Time: 4	23472	415	0	22	20	0	1	Time: 0
51	49	1	1742	*****	23473	416	0	23	21	1	2	*****
52	50	0	1743	1 1	23474	417	0	24	22	1	3	1 1
53	51	0	1744	2 1	23475	418	0	25	23	1	4	2 1
54	52	0	1745	3 1	23476	419	0	26	24	0	5	3 1
55	53	1	1746	4 0	23477	420	1	27	25	0	6	4 0
56	54	1	1747	5 0	23478	421	1	28	26	0	7	5 0
57	55	1	1748	6 0	23479	422	1	29	27	1	8	6 0
58	56	0	1749	7 0	23480	423	1	30	28	1	9	7 0
59	57	0	1750	8 1	23481	424	1	31	29	1	0	8 1
60	58	0	1751	9 1	23482	425	1	32	30	0	1	9 1
61	59	0	1752	10 1	23483	426	1	33	31	0	2	10 1
62	60	1	1753	11 0	23484	427	1	34	32	0	3	11 0
63	61	1	1754	12 0	23485	428	1	35	33	0	4	12 0
64	62	1	1755	13 0	23486	429	0	36	34	1	5	13 0
					23487	430	0	37	35	1	6	14 1
					23488	431	0	38	36	1		
					23489	432	1					